

CSE 220 – Signals and Linear Systems

Practice Assignment: Signal Transformations & Properties
January 2026 | Total Marks: 50 | Estimated Time: 2 Hours

The base signal used throughout this assignment is defined as: $x(t) = e^{-|t|}$, bounded between $-2 \leq t \leq 2$ (values outside this range are 0). A template Python file (*practice_set.py*) accompanies this sheet. Implement every marked function; do not modify the `main()` dispatcher or the `base_signal()` helper.

Question 1 – Time Reversal & Amplitude Scaling [10 Marks]

Objective

Implement a function that reverses a signal in time and scales its amplitude, then visualise both the original and the transformed signal.

Background

Given a continuous-time signal $x(t)$, the time-reversed and amplitude-scaled version is:

$$y(t) = \alpha \cdot x(-t)$$

Because our signal is sampled on a finite grid, time-reversal is achieved via *interpolation*: build an interpolator from the (t, x) samples, then evaluate it at $-t$. Points outside the original range return 0.

Tasks

- ★ Implement `q1_transform(t, x, alpha)` that returns $\alpha \cdot x(-t)$.
- ★ Build an interpolator (use `scipy.interpolate.interp1d`) and evaluate at $-t$.
- ★ Multiply the result by `alpha`.
- ★ The infinite loop in `q1_main()` reads α from stdin; enter 'q' to quit.
- ★ Plot $x(t)$ and $y(t)$ on the same figure with correct labels, legend, and grid.

Verification examples (use $\alpha = 1$ to check reversal only)

t	x(t)	y(t) = 1·x(-t)
0	$e^0 = 1.000$	1.000
1	$e^{-1} \approx 0.368$	0.368 (== x(-1))
-1	$e^{1} \approx 0.368$	0.368 (== x(1))
2.5	0 (out of range)	0

Criterion	Marks
Correct use of interpolation for time reversal	4
Correct amplitude scaling by alpha	4
Infinite loop with quit on "q"	2

Question 2 – Time Scaling [10 Marks]

Objective

Implement time-scaling of a sampled signal and visualise the compression and stretching effects.

Background

Replacing t with $a \cdot t$ inside a signal argument compresses or stretches it in time:

$$y(t) = x(a \cdot t)$$

- $|a| > 1$: signal is *compressed* in time (faster).
- $0 < |a| < 1$: signal is *stretched* in time (slower).
- $a < 0$: time-compression/stretching *combined with reversal*.
- $a = 0$: undefined — reject this input.

Tasks

- ★ Implement **q2_transform(t, x, a)** that returns $x(a \cdot t)$.
- ★ Build an interpolator and evaluate it at $a \cdot t$.
- ★ Reject $a = 0$ with a descriptive message; do not crash.
- ★ The infinite loop in **q2_main()** reads **a** from stdin; enter 'q' to quit.
- ★ Plot $x(t)$ and $y(t)$ with labels, legend, and grid.

Hint: the t axis in `q2_main()` is wider (-4 to 4) so you can observe stretching beyond the original support.

Criterion	Marks
Correct interpolation and evaluation at $a \cdot t$	4
Correct handling of compression vs. stretching	3
Rejection of $a = 0$ and infinite loop with quit	3

Question 3 – Combined Signal Transformation [10 Marks]

Objective

Apply all three elementary transformations — amplitude scaling, time scaling, and time shifting — in a single step.

Background

The general linear transformation of a continuous-time signal is:

$$y(t) = \beta \cdot x(a \cdot t - t_0)$$

where β scales amplitude, a scales (and possibly reverses) time, and t_0 shifts the argument. The order of evaluation inside the argument is: first scale, then shift.

Tasks

- ★ Implement **q3_transform(t, x, beta, a, t0)** that returns $\beta \cdot x(a \cdot t - t_0)$.
- ★ Build a single interpolator and evaluate at $a \cdot t - t_0$; multiply by β .
- ★ **q3_main()** reads three space-separated values *beta a t0* per iteration.
- ★ Validate: reject $a = 0$; handle all other float inputs.
- ★ Plot $x(t)$ and $y(t)$ with proper annotation in the legend showing β , a , and t_0 .

Example: input '2 0.5 1' should yield $y(t) = 2 \cdot x(0.5t - 1)$ — a stretch-then-shift with doubled amplitude.

Criterion	Marks
Correct combined evaluation at (a*t - t0)	5
Correct amplitude scaling by beta	3
Input parsing, validation, loop, and plot labels	2

Question 4 – Odd / Even Decomposition [10 Marks]

Objective

Decompose any signal into its even and odd components and verify that their sum reconstructs the original.

Background

Every signal $x(t)$ can be uniquely decomposed as:

$$x_e(t) = [x(t) + x(-t)] / 2 \text{ — even part } (x_e(-t) = x_e(t))$$

$$x_o(t) = [x(t) - x(-t)] / 2 \text{ — odd part } (x_o(-t) = -x_o(t))$$

$$x(t) = x_e(t) + x_o(t) \text{ — reconstruction}$$

Tasks

- ★ Implement **q4_even(t, x)** returning $x_e(t)$.
- ★ Implement **q4_odd(t, x)** returning $x_o(t)$.
- ★ Both functions must use interpolation to evaluate x at $-t$.
- ★ **q4_main()** runs once and produces a 2x2 subplot: $x(t)$, $x_e(t)$, $x_o(t)$, and $x_e(t)+x_o(t)$ (verify this equals $x(t)$).
- ★ Confirm numerically: $\max|x - (x_e + x_o)|$ should be $< 1e-10$.

Property check: for our base signal $x(t) = e^{-|t|}$ (which is already even), $x_o(t)$ should be identically 0 and $x_e(t) = x(t)$.

Criterion	Marks
Correct even part using interpolation at $-t$	4
Correct odd part using interpolation at $-t$	4
2x2 subplot with reconstruction panel	2

Question 5 – Energy & Power [10 Marks]

Objective

Numerically estimate the energy and average power of a signal using Riemann summation and print/annotate the results.

Background

The total energy and average power are defined as:

$$E = \int |x(t)|^2 dt \approx \sum |x(t_i)|^2 \cdot \Delta t$$

$$P = (1/T) \cdot \int_{-T/2}^{T/2} |x(t)|^2 dt \approx E / T_{\text{total}}$$

where Δt is the sampling interval ($t[1] - t[0]$) and $T_{\text{total}} = t[-1] - t[0]$.

Tasks

- ★ Implement **q5_energy(t, x)**: return $\sum |x(t_i)|^2 \cdot \Delta t$.
- ★ Implement **q5_power(t, x)**: return energy divided by T_{total} .
- ★ **q5_main()** prints the numerical values and plots $x(t)$ with a shaded $|x(t)|^2$ region to visualise energy density.
- ★ Annotate the plot with the computed E and P values (see template's `gcf().text` call).
- ★ Discussion (written in a comment in the code): Is this signal an energy signal or a power signal, and why?

Expected result: Since $x(t)$ has finite support and finite amplitude, E is finite and $P \rightarrow 0$ as $T \rightarrow \infty$. Your numerical P will be small but nonzero because the window is finite.

Criterion	Marks
Correct Riemann-sum energy calculation	4
Correct average power calculation over the window	4
Plot with shading, annotations, and printed values	2

General Instructions

- Only modify functions marked with # **TODO: implement**.
- Do not import additional libraries beyond numpy, scipy, and matplotlib.
- Each question's `main()` can be run independently from the menu.
- All plots must include: title, x-label, y-label, legend, and grid.
- Partial credit is awarded for correct logic even if the output is not fully correct.

Total: 50 Marks | Each question is worth 10 marks.